

Adaptive progressive verification with short private check banks

9 October 2026

Abstract

We give a bounded-reuse progressive verifier for signature checks consisting of an auxiliary predicate and a finite-field linear equation. The verifier samples fresh indices from a short private bank of preprocessed rows. Previous queries may adaptively reveal row indices and zero/nonzero answers. A first-spoil tail bounds how many rows can vanish on a subsequently chosen nonzero residual: after at most L previous row queries, the exceptional probability is at most $\binom{L+s}{u}/(q-L-s)^u$, provided $L+s < q$. A designated final residual and threshold committed before fresh index sampling therefore have invalid-acceptance probability at most $\mathbb{E}[(u-1)/s]^T + E_{\max}\delta$ over a deterministic maximum of $E_{\max}$ epochs. We prove a separate joint-threshold bound, preserve the original auxiliary checks, and account for field arithmetic in initialization, incremental matrix preparation, and the final invocation across actual buffer promotions. A finite example exhibits an initialization-inclusive field-multiplication crossing at early nominal confidence levels against specified terminal-compression and code-sampling baselines. It does not establish wall-clock or whole-resource superiority. The finite compiler, trace-erasure refinement and parameter certificates are formalized in Lean. The contribution is the combined quantitative verification method; excluded-key conditioning and random-check sampling are established ingredients.

1 Introduction

A progressive verifier gives a useful partial answer before a complete verification finishes. For signatures whose verification includes a large matrix-vector product, private preprocessing can replace an online matrix computation by a small number of stored row checks. Reusing those rows is attractive, but previous verification responses reveal information about them. A confidence curve that ignores this interaction can be unjustified even when a single fresh check has a simple error bound.

We analyze a particular arrangement of known ingredients. Independently sampled secret rows are preprocessed once per bounded epoch. Each progressive step samples a fresh index from that bank. The analysis permits some rows to have experienced previous false acceptances; it controls their number rather than requiring that no previous error has occurred. This change permits a useful finite arithmetic regime with a short bank, a small scheduled preparation charge, and an explicit small additive history term.

The guarantee concerns an adaptive prior-query experiment followed by one designated final invocation. Its residual and interruption threshold are committed before that invocation's fresh random indices. The guarantee is an aggregate probability bound, together with a joint event bound for each threshold. It is not a per-transcript posterior guarantee, and it does not support choosing an earlier accepted response retrospectively as the final invocation.

Our contribution is a bounded-reuse compiler theorem and a precisely scoped arithmetic tradeoff. We do not claim a new signature primitive, a new probability inequality, a first general feasibility result for progressive verification, or a practical speedup. In particular, fixed terminal compression remains substantially better when early low-confidence outputs are not useful.

Formal artifact. The accompanying revision consists of 67 Lean modules. A separate source-only rebuild checked all modules and audited 862 theorem declarations; only the standard Lean axioms `propext`, `Classical.choice` and `Quot.sound`, or subsets, occur. A separate semantic review passed the finite ideal-sampling, serialized-invocation scope stated here. These checks establish the specified mathematics, not originality, implementation safety, or source-signature hardness. Appendix A maps the paper to the artifact.

2 Setting and experiment

Let $\mathbb{F}$ be a finite field with q elements. Let $A \in \mathbb{F}^{n \times m}$, with $n > 0$. A candidate x determines vectors $\sigma(x) \in \mathbb{F}^m$ and $h(x) \in \mathbb{F}^n$, together with a Boolean auxiliary predicate $\text{Aux}(x)$. Define

$$w(x) = A\sigma(x) - h(x).$$

The supplied ordinary verifier accepts exactly when $\text{Aux}(x)$ is true and $w(x) = 0$. For a signature application, x contains the message and signature; Aux retains the original parsing, norm, and other required checks. The theorem is independent of how a signer generates valid candidates and of how h is computed.

Definition 2.1 (Row-bit experiment). A bank consists of independent uniform rows $C_1, \dots, C_s \in \mathbb{F}^n$. A deterministic observer is a finite binary decision tree. At each internal node it chooses an index $j \in \{1, \dots, s\}$ and a vector $v \in \mathbb{F}^n$, and receives the bit

$$b = \mathbf{1}[\langle C_j, v \rangle = 0].$$

The choices may depend on all previous indices and bits. The tree has depth at most L . At a leaf it outputs a residual w ; in the primitive experiment, every such output is nonzero. The observer does not receive nonzero field values or any additional information about the bank.

This oracle is stronger than an atomic signature-verification oracle that reports only rejection or a confidence value. It even allows the observer to choose prior row indices and change the vector adaptively within a batch. Proving security for this stronger interface covers its restricted use by the compiler. It does not cover arbitrary leakage of stored coefficients or field-operation timing.

Definition 2.2 (Designated final invocation). After its prior interaction, the observer fixes a candidate x and a threshold T . Then the verifier draws T independent uniform indices from $\{1, \dots, s\}$, with replacement. It accepts the projected equation if all selected rows vanish on $w(x)$. A final signature acceptance also requires $\text{Aux}(x)$.

The final fresh index function is sampled after x and T have been fixed. It is not an earlier response chosen from the transcript. The formal finite model allows T to be any precommitted natural number; the public progressive interface uses $T = \min\{k, \text{requested}\}$.

Epochs and external randomness. There is a deterministic maximum allowance $E_{\max}$ of epochs. Each epoch has a fresh independent bank and at most R reserved invocations, each with at most k prior row checks. We use the conservative cap $L = Rk$. Every attempted invocation consumes a slot, including an empty or interrupted one. The adversary may stop before the last allowed epoch. A continuing epoch uses all R slots; a stopping epoch leaves a slot for the designated final invocation. The final slot is charged as well. For a budget of at most Q prior reserved invocations, $E_{\max} = \lfloor Q/R \rfloor + 1$ is a conservative deterministic allowance including the designated final epoch.

Randomized observers are finite mixtures of deterministic observers with nonnegative rational weights summing to one. Their random tapes are independent of the bank samples. This includes fixed bounded random tapes for the adversary and its environment. The expectation in our final bound uses the actual resulting distribution of T ; it is not replaced by a fixed threshold.

Relationship to progressive-verification syntax. The stateful progressive interface and setup-inclusive accounting follow the setting of Boschini et al. [3], Definitions 3, 5, 7 and 10. We state the random-threshold guarantee explicitly to avoid interpreting it as conditional calibration. State sequences must be generated by the specified transitions: restarting a cold state before every call is not the same amortized execution.

3 The private-bank compiler

For a secret row C_j , preprocess

$$Z_{j,a} = \sum_{i=1}^n C_{j,i} A_{i,a}, \quad 1 \leq a \leq m.$$

A stored check on candidate x evaluates

$$\sum_{a=1}^m Z_{j,a} \sigma_a(x) - \sum_{i=1}^n C_{j,i} h_i(x) = \langle C_j, w(x) \rangle. \quad (1)$$

The checking algorithm reads Z , C and the candidate vectors; it does not compute the full residual or multiply A by σ online. The public matrix A remains available for preparing later banks.

Set $d = m + n$, $B = snm$, and choose positive R, W satisfying $B = RW$. The checked scheduling theorem uses this exact divisibility condition. Maintain a current bank and its complete stored matrix, and an independent next bank with a partially prepared matrix. Initially prepare the current matrix completely and initialize the next accumulator. Before responding to each invocation, advance the next matrix accumulator by exactly W multiply-add tasks and reserve one slot. After R invocations, the next matrix is complete. On continuing to another epoch, promote that actual matrix, draw a new lookahead bank, and reset the preparation cursor.

A task is a triple (j, a, i) and adds $C_{j,i} A_{i,a}$ to accumulator entry (j, a) . A mixed-radix cursor enumerates the snm tasks once each. No list of all tasks is retained by the operational implementation. The final invocation is a separate call: it advances preparation, consumes one reservation, and executes every one of its T requested checks. It does not reveal an early, value-dependent final rejection time.

Scheduling pseudocode. The following describes one atomic serialized invocation; its preparation phase must complete before a confidence response is returned.

1. At setup, draw current rows C and lookahead rows D , initialize stored accumulators Z, P to zero, compute $Z = CA$, and set the lookahead task cursor and local slot counter to zero.
2. On every invocation, reserve one slot without refund. Execute the next W cursor tasks for $P = DA$, even if the requested threshold is zero or the later progressive part is interrupted.
3. Set $t = \min\{k, \text{requested}\}$ and $\text{ok} = \text{true}$. For each of the t steps, sample a fresh uniform index j , execute (1) using (C, Z) , and conjoin the zero-test bit into ok . Execute all requested final checks rather than reporting an earlier private-bit-dependent failure time.
4. Return reject if $\text{Aux}(x)$ or ok is false; otherwise return $1 - \rho^t$.
5. Before admitting the next invocation after a full R -slot epoch, promote $(C, Z) \leftarrow (D, P)$, draw the next independent lookahead bank, reset its accumulator and cursor, and reset the local slot counter. Count this boundary's sampling and initialization work; no idle period or separate processor is assumed free.

Empty and aborted progressive batches occupy their reserved slot. Faults or interruption inside the atomic preparation/reset operation are outside the stated model, rather than a reason to refund the charge.

For $1 \leq u \leq s$, let

$$\rho = \frac{u-1}{s}, \quad \alpha(t) = 1 - \rho^t. \quad (2)$$

If $\text{Aux}(x)$ holds and all projected checks pass, the interface returns $\alpha(T)$; otherwise it rejects. At $T = 0$, it returns confidence zero when the auxiliary check passes. The curve is nondecreasing, lies in $[0, 1]$, and has $\alpha(0) = 0$ and $\alpha(1) > 0$. Request clamping preserves monotonicity.

Proposition 3.1 (Perfect completeness). *If $\text{Aux}(x)$ is true and $w(x) = 0$, every requested sequence of fresh indices accepts and returns $\alpha(T)$.*

Proof. Equation (1) is zero for every row. The conjunction of all T tests is true, including the empty conjunction at $T = 0$. The original auxiliary predicate also passes. The operational refinement establishes that every promoted stored matrix still satisfies (1). $\square$

4 The first-spoil tail

We give the finite probability argument in full. It uses established excluded-key conditioning followed by a multi-row tail argument. A row is *spoiled* when it first answers zero to a nonzero queried vector. Later behavior of that row does not add another first-spoil event.

Lemma 4.1 (A nonzero linear functional). *For $v \in \mathbb{F}^n \setminus \{0\}$, the map $c \mapsto \langle c, v \rangle$ is onto $\mathbb{F}$, and each fiber contains q^{n-1} rows. In particular, a uniform row satisfies $\langle c, v \rangle = 0$ with probability $1/q$.*

Proof. Choose a coordinate i with $v_i \neq 0$. For arbitrary values of the other $n-1$ coordinates and any desired output $a \in \mathbb{F}$, exactly one value of c_i solves the equation: divide $a - \sum_{\ell \neq i} c_\ell v_\ell$ by v_i . Thus every output has the same number of preimages. $\square$

Lemma 4.2 (Conditional row cells). *Fix an observer's coins and a feasible complete transcript prefix. For each row j , let V_j be the set of rows satisfying the equations and inequations recorded for that row in the prefix. The compatible bank set is exactly $\prod_j V_j$, and the conditional bank distribution is uniform on this product.*

Proof. The fixed decision tree and the recorded bits determine every query along the prefix, including its index and vector. A bank follows this path if and only if it gives each recorded answer. Grouping those conditions by row gives precisely the product set. The initial distribution gives equal mass to every bank; conditioning on a nonempty subset therefore gives the uniform distribution on that subset. Since the subset is a Cartesian product, its row marginals are uniform on the corresponding cells. This reasoning fixes the transcript before grouping constraints; it does not assume that adaptively chosen query vectors were fixed independently of the bank. $\square$

Lemma 4.3 (First-spoil probability). *Suppose a row is unspoiled after a transcript containing at most M queries, where $M < q$. For the next query, the conditional probability that this row becomes newly spoiled is at most $1/(q - M)$. A query of the zero vector or of an already spoiled row contributes no new event.*

Proof. An unspoiled row has no previous zero answer on a nonzero vector. Zero-vector queries impose no restriction. Every remaining recorded restriction excludes the kernel of one nonzero linear functional. If there are $\ell \leq M$ such restrictions, the allowed cell has size at least

$$q^n - \ell q^{n-1} = (q - \ell)q^{n-1}.$$

For any next nonzero vector, at most q^{n-1} allowed rows answer zero. The conditional probability is therefore at most $1/(q - \ell) \leq 1/(q - M)$. The preceding lemma justifies applying this row count after an adaptive full transcript. Empty conditional cells need not be assigned probabilities; the formal proof instead uses the equivalent joint cardinal inequality. $\square$

Lemma 4.4 (Adaptive finite tail). *Let $Y_1, \dots, Y_M$ be zero-one variables revealed sequentially. Let $0 \leq p \leq 1$, and suppose each conditional probability of $Y_i = 1$, given a feasible preceding history, is at most p . Then, for every integer $u \geq 1$,*

$$\Pr \left[\sum_{i=1}^M Y_i \geq u \right] \leq \binom{M}{u} p^u.$$

Proof. For any fixed u positions, reveal them in increasing order. Repeated conditional summation bounds the probability that all their indicators equal one by p^u . If at least u indicators are one, some u -element subset of positions has this property. A union bound over the $\binom{M}{u}$ subsets proves the claim. No independence of the indicators is required. The Lean proof states this argument as an integer cardinal inequality on finite decision-tree cells, so it uses no analytic conditional-expectation machinery. $\square$

Theorem 4.5 (Adaptive final zero-row bound). *Let an observer make at most L row-bit queries and then output a nonzero residual w . Put $M = L + s$, assume $M < q$, and let $1 \leq u \leq s$. If*

$$K(C, w) = \#\{j : \langle C_j, w \rangle = 0\}, \quad \delta = \frac{\binom{L+s}{u}}{(q - L - s)^u},$$

then $\Pr[K(C, w) \geq u] \leq \delta$.

Proof. After the observer commits to w , append one virtual test of w against each row. These tests are only a proof device. The augmented adaptive experiment has at most $L + s = M$ queries; pad shorter paths with queries that cannot create first-spoil events.

For every row that vanishes on the final w , either that row was already spoiled or its virtual query newly spoils it. Since $w \neq 0$, these alternatives cover all final-zero rows. Different rows require different first-spoil events. Consequently $K(C, w) \geq u$ implies at least u first-spoil events in the augmented experiment.

Apply Lemma 4.3 with the conservative total budget M at every step. Each indicator of a new first-spoil event has conditional probability at most $1/(q - M)$. Lemma 4.4 gives the stated bound. The argument does not try to describe a row's distribution after its first spoil, and does not require every row to remain unspoiled. $\square$

Denominator scope. The checked endpoint deliberately uses $q - L - s$. Tracking negative restrictions separately can suggest sharper expressions, but no sharper denominator is needed for our finite example or claimed formal result. We do not silently substitute $q - L$ in parameter certificates.

5 Progressive soundness and epochs

Lemma 5.1 (Fresh sampling). *For a fixed zero-row set $S \subseteq \{1, \dots, s\}$ with K members and a fixed $t \geq 0$, the probability that all t independent uniform indices lie in S is $(K/s)^t$.*

Proof. There are s^t index functions from $\{1, \dots, t\}$ to the bank and exactly K^t whose values all lie in S . Their ratio gives the result. Both sets contain one empty function when $t = 0$. $\square$

Theorem 5.2 (Progressive compiler bound). *Consider the compiler of Section 3 in the experiment of Section 2. Let $E_{\max}$ be a deterministic maximum number of independent epochs, let $L = Rk$, and assume $L + s < q$ and $1 \leq u \leq s$. Set $\rho = (u - 1)/s$ and $\delta = \binom{L+s}{u}/(q - L - s)^u$. Let IA denote acceptance in the designated final invocation when $\text{Aux}(x)$ is true and $w(x) \neq 0$. Then*

$$\Pr[\text{IA}] \leq \mathbb{E}[\rho^T] + E_{\max}\delta, \quad (3)$$

$$\Pr[\text{IA} \wedge T = t] \leq \rho^t \Pr[T = t] + E_{\max}\delta \quad \text{for every } t \geq 0. \quad (4)$$

The same statements hold after averaging independent finite rational observer tapes.

Proof. For a single epoch, define its bad event to be that its locally chosen nonzero final residual has at least u zero rows. If a candidate has zero residual or fails an application gate, it need not contribute an invalid-acceptance event. Formally, zero-residual leaves may be mapped to an arbitrary fixed nonzero dummy vector while the corresponding invalid-acceptance gate is false; this leaves every counted real event unchanged.

Condition on the incoming observable history before an epoch. Its bank is still uniform and independent. Theorem 4.5 bounds that epoch's bad probability by δ . The candidate at an epoch is determined by its preceding history and its own bank interaction; it cannot be revised using a later bank. A union bound over the fixed maximum list of $E_{\max}$ epochs bounds the probability of any relevant bad epoch by $E_{\max}\delta$. An adaptive earlier stopping rule selects one of these locally fixed candidates. We use the deterministic allowance here, not the realized or expected number of reached epochs.

Let **Bad** denote the resulting union event. Outside **Bad**, on a branch eligible for **IA**, the selected nonzero residual has $K \leq u - 1$ zero rows. Conditional on the complete state before fresh final sampling, the probability of **IA** is therefore at most ρ^T . On a gated-off branch it is zero; in particular,

a valid zero residual has $K = s$ but contributes no invalid-acceptance event. This remains true for a state-dependent T , because the fresh indices are sampled after T is chosen. Thus

$$\Pr[\text{IA}] \leq \mathbb{E}[\mathbf{1}_{\neg \text{Bad}} \rho^T] + \Pr[\text{Bad}] \leq \mathbb{E}[\rho^T] + E_{\max} \delta.$$

Restrict the same finite sum to $T = t$ to obtain

$$\Pr[\text{IA} \wedge T = t] \leq \rho^t \Pr[T = t \wedge \neg \text{Bad}] + \Pr[\text{Bad} \wedge T = t],$$

which implies (4). Finally, multiply each fixed-tape bound by its nonnegative tape weight and sum. Normalization leaves one copy of the common $E_{\max} \delta$ term and preserves the actual threshold expectation or threshold-event mass. $\square$

Remark 5.3 (Meaning of confidence). Dividing (4) by a very small $\Pr[T = t]$ enlarges its additive term. Neither (3) nor (4) asserts that every transcript, or every rare selected threshold, has posterior error at most $\rho^t + E_{\max} \delta$.

Corollary 5.4 (Separate source-signature term). *Suppose the probability of producing an eligible fresh candidate satisfying the original auxiliary predicate and original equation, in the same surrounding interaction, is at most ε_{src} . Then progressive acceptance of an eligible fresh candidate has probability at most*

$$\varepsilon_{\text{src}} + \mathbb{E}[\rho^T] + E_{\max} \delta.$$

Proof. Split the event according to whether the original equation holds. The equation-valid portion is bounded by the stated source assumption; the equation-invalid portion is a subset of IA . Apply Theorem 5.2. $\square$

The artifact names this premise `OrdinarySourceBound`. It is not a proved reduction to a particular EUF game or an instantiated lattice-hardness bound. Such a source-scheme instantiation must supply the appropriate simulation and security assumption separately.

6 Operational refinement

The probability experiment is connected to actual stored computations rather than used as an assumed oracle specification. The refinement has four parts.

First, the stored checker computes both dot products in (1). Each has a zero-initialized accumulator. Induction over its coordinates proves its value and operation counters. Induction over an adaptive query tree then proves that actual stored bits choose exactly the same branches as the residual-row experiment, preserving the output and all observed bits.

Second, the preparation cursor visits each task (j, a, i) exactly once. Its matrix entry after a prefix is the sum of exactly the processed products for that entry. After $B = snm$ tasks, every entry equals $(CA)_{j,a}$. Composition of cursor prefixes shows that R reserved invocations at W tasks each produce the next stored matrix when $RW = B$.

Third, a continuing phase promotes the accumulator it actually computed, not a separately supplied ideal matrix. It resets its next cursor and uses the new stored rows for subsequent checks. A stopping phase returns the current bank and matrix without promoting. An induction over the maximum phase list proves that the actual output, selected epoch, current bank and local reservation count equal those in the raw stopped experiment. The active matrix satisfies (1) at every

such boundary. Because the visible output of an epoch does not depend on its hidden lookahead preparation, prefetch is observationally equivalent to deferring a bank sample until its activation. The product-sample proof then supplies the required independent uniform current bank.

Fourth, the final invocation uses the retained actual matrix, reserves its slot and advances the actual pending cursor. Its accepting fresh index functions are exactly the all-hit set from the fresh-sampling lemma, gated by Aux and, for the error event, by $w(x) \neq 0$. This proves equality of the operational error mass and the mass bounded in Theorem 5.2, including $T = 0$ and auxiliary rejection.

The instrumented proofs retain traces and intermediate records for reasoning. Independently defined trace-free runners retain scalar check counts instead. Whole-record erasure equalities preserve outputs, matrices, cursors, stopping decisions, counters and acceptance. The final erased result also discards the old pending-buffer copy: it retains only the updated pending matrix. This is essential for the claimed field-state inventory.

7 Resource accounting

Theorem 7.1 (Exact field arithmetic). *Let a valid execution contain N invocations in total, including the designated final call, and S executed stored-row checks. Assuming $B = RW$, its initialization-inclusive number of field multiplications is*

$$\mathcal{M} = B + NW + dS. \quad (5)$$

The same count applies to accumulator additions. Stored checks additionally perform S subtractions. These formulas include unused next-bank preparation, aborted invocations and threshold-zero calls.

Proof. Initialization executes exactly B cursor tasks. Each reservation advances the pending cursor by W tasks; the capacity invariant ensures these are real tasks rather than steps past an already finished bank. Thus preparation contributes exactly $B + NW$ products and additions. A stored row performs $m + n = d$ products and additions and one subtraction. Summing over the actual S checks gives (5). The final call is included through its extra reservation and the actual difference of its pending cursor counters. The erased runner has identical arithmetic counters. $\square$

Epoch count. If e promotions occur before the selected stopping epoch and a prior invocations occur within that epoch, the actual prior count is $eR + a$; the final call makes $N = eR + a + 1$. This links a predetermined invocation budget to a predetermined sufficient maximum epoch allowance. It does not replace $E_{\max}$ in Theorem 5.2 by the random realized count $e + 1$.

Ideal randomness. The eager-lookahead schedule reads two independent banks at setup and one more at each promotion. Its bank-sampling resource is therefore exactly $(e + 2)sn$ ideal uniform field draws. A promotion may entail an sn -draw burst. Each randomized stored check also needs a row-index draw; the final call uses exactly T independent indices. The Lean sampling model is finite and ideal. It does not implement an entropy source, a bit-level rejection sampler, or a pseudorandom generator.

Field coordinates and other memory. The erased persistent checking state has two $s \times n$ row banks and two $s \times m$ stored matrices. Including retained public A , its field-coordinate inventory is

$$nm + 2s(n + m). \quad (6)$$

Scalar counters, an acceptance bit and the external history/interface remain separate. The inventory excludes caller-owned candidate vectors, adversary state and program, and the mathematical random-tape specification. It is not a compiled heap/stack bound; integer counter bit lengths and a concrete array representation still matter.

Writes, allocation and decoding are not free. The cursor uses two integer divisions and two remainders to decode each task, together with loop and indexing operations. Each task updates an accumulator entry. A conventional dense-array realization of the abstract zero accumulator additionally zero-fills sm cells at initialization or reset. With two initial matrices and e subsequent resets, that realization performs $(e + 2)sm$ zero-fill writes, in addition to the cursor’s $B + NW$ updates and other reads/writes. In particular, a reset can have a substantial memory-write burst. These zero-fill counts describe that straightforward array realization; they are not machine-code cost counters in the Lean field-arithmetic theorem. An overwrite-on-first-product optimization could avoid a separate fill, but it is not the frozen implementation analyzed here.

Allocation, pointer management and memory bandwidth therefore remain implementation obligations. Together with sampling bursts, they prevent interpreting the preparation schedule as a deamortization of *all* work. We prove scheduled field arithmetic, not worst-case wall-clock checkpoint latency. Original hashing, parsing and norm checking must be added unchanged. A practical benefit is an open empirical question.

8 A certified finite arithmetic crossing

The following is a verifier-arithmetic point, not a recommended signature parameter set:

Field order q	1073741789	Public rows n	1024
Public columns m	61852	Bank rows s	64
Epoch invocations R	262144	Maximum depth k	128
Bad-row threshold u	33	Tasks per call W	15463

The field order is proved prime, and the artifact instantiates $\mathbb{Z}/q\mathbb{Z}$. Here $d = 62876$, $L = 33554432$, and

$$B = 4053532672 = RW, \quad \rho = \frac{1}{2}, \quad \delta < 2^{-286}.$$

For a deterministic allowance $E_{\max} \leq 2^{64}$, the checked certificate gives $E_{\max}\delta < 2^{-222}$. The field-state inventory is 71384576 elements, including A . It is larger than the original public matrix alone; the word “short” refers to the private row bank, not a reduction of total retained public-key storage.

8.1 The terminal comparator

We derive a fixed high-confidence terminal comparator in the same linear interface. Preprocess seven independent private projection rows and test the complete block at each high-confidence verification. Before the full block finishes, return confidence zero without exposing partial-row answers or early data-dependent rejection. Its ordinary-invalid final acceptance over at most Q prior completed queries is bounded by $(Q + 1)/q^7$ using a standard first-error coupling. Fix the external random tape and follow an ideal ordinary verifier that accepts exactly the valid equations passing **Aux** and rejects all others. This ideal path is independent of the compression key. Along it there are at most $Q + 1$ fixed nonzero residuals on auxiliary-eligible queries. Each is annihilated by all seven

independent rows with unconditioned probability q^{-7} . Union-bound these events. Until the first such event, the real complete-block answers agree with the ideal path; valid and auxiliary-failing inputs give key-independent answers. This proves the bound without asserting that the real key stays uniform after conditioning on previous rejections. Averaging the independent external tape preserves it.

For $Q \leq 2^{64}$ the exact integer certificate gives $(Q + 1)/q^7 < 2^{-145}$. Setup uses $7nm$ products, online completion uses $7d$, and the stored linear representation uses $7d$ field elements if A can be discarded. This is a derived fixed-compression baseline, not a verbatim parameter claim from another paper. It has no nontrivial early confidence before completing seven rows. Revealing its partial-row outcomes would be a different interface and would not inherit this first-error proof.

For the pool, if $N \geq R$ and $\bar{t} = S/N$, Theorem 7.1 implies

$$\frac{\mathcal{M}}{Nd} \leq \bar{t} + \frac{2W}{d} < \bar{t} + 0.491857.$$

The strict comparison is kernel checked for actual executions: if $S \leq 6N$, then $\mathcal{M} < 7Nd$, even granting the terminal comparator free preprocessing. Thus the pool reaches the first six nominal confidence levels $1 - 2^{-t}$ with a smaller initialization-inclusive average field-multiplication budget than this specified seven-row comparator’s completed block. Its actual fixed-threshold error cap is $2^{-t} + E_{\max}\delta$, including the certified slack $E_{\max}\delta < 2^{-222}$. At high confidence, the terminal method wins decisively, as it does in storage and cold setup. This is a crossing of specified arithmetic curves, not dominance.

Shorter terminal blocks materially narrow the comparison. Seven rows are appropriate for the stated high-confidence comparator, but they are not the best fixed block for every weak target. At the same $Q \leq 2^{64}$, a three-row terminal block already has fixed-threshold total error

$$(Q + 1)/q^3 < 2^{-25}$$

and completes after $3d$ products. A direct integer inequality verifies this value; a 2^{-26} upper bound would fail at the displayed maximal Q . Since $2W < d$, only the pool’s first two nominal checkpoints precede this three-row block under the same $N \geq R$ initialization-inclusive average-product comparison. The six-checkpoint theorem against seven rows remains true, but does not describe a frontier over terminal compression methods.

Independent short and long terminal blocks can also be staged to offer an intermediate endpoint and a later stronger one. Their setup, row counts, observable outputs and history budgets must then be analyzed together. In particular, the three-row first-error budget is much larger than the pool’s additive slack; the unconditional fixed-threshold estimate alone does not establish the same rare-threshold joint calibration. We claim neither optimality nor exclusion of such alternatives. The demonstrated early crossing against the three-row comparator concerns only the first two nominal checkpoints, and practical value remains unestablished.

8.2 A code-sampling comparator

Taleb and Vergnaud use a full-distance expanded representation and sampled checks [2]. For an explicit derived comparator, use Reed–Solomon evaluation of residual polynomials of degree at most 1023, retain the evaluated public matrix, use an implicit generator, and discard A . Give this

comparator the entire pool field-state budget and, favorably, no setup or scratch charge. At most

$$\ell = \left\lfloor \frac{71384576}{61852} \right\rfloor = 1154$$

stored rows fit. A nonzero residual polynomial has at most 1023 zero coordinates. For $0 \leq t \leq 1154$, uniform distinct row sampling therefore has worst-case error

$$\frac{\binom{1023}{t}}{\binom{1154}{t}} = \frac{(1023)_t}{(1154)_t}.$$

The finite products certify the following first six nominal/main-term thresholds. For the pool, the actual fixed-threshold error bound is $2^{-b} + E_{\max}\delta$, not exactly 2^{-b} . RS needs no such history slack. Thus the table aligns nominal checkpoints rather than silently asserting exact equal-error guarantees:

Nominal confidence $1 - 2^{-b}$, b	1	2	3	4	5	6
Pool row checks	1	2	3	4	5	6
RS distinct row checks	6	12	18	23	29	34
Three-row terminal completion	3	3	3	3	3	3
Fixed seven-row high-confidence block	7	7	7	7	7	7

The pool additionally pays its scheduled and initial preparation. The arithmetic comparison uses a conventional full stored-row dot product, including the corresponding target evaluation. It is not a lower bound on arbitrary circuits, batched verification, alternative codes or all implementations. Reed–Solomon coding facts are prior mathematical premises; the artifact certifies the displayed finite threshold and storage arithmetic, not an entire RS implementation.

8.3 Comparison dimensions

The pool requires $2sn$ initial ideal field draws and sn per promotion; the seven-row terminal representation requires $7n$ at setup and no refresh within its stated lifetime. The deterministic RS comparator has no secret-bank sampling, but has actual encoding, target evaluation and indexing costs. Dense accumulators require initialization and writes in every method, according to their chosen implementation. These resources cannot be silently priced at zero to claim a runtime improvement. The pool’s initial product cost alone exceeds four billion at this example point. Whether early confidence is useful enough to justify the retained state, recurring work and bursts depends on a workload not established here.

9 Prior work and contribution boundary

Fischlin introduced progressive verification and studied prior-query and committed final-attempt experiments, including randomized checking order [1]. Early confidence and final precommitment are therefore not new. Nguyen and Roscoe’s sampled-digest work is also a conceptual antecedent for checking a random part of a bank with a bounded number of problematic entries [6]; its published signed material and guarantee differ from our private row-bit experiment.

Boschini et al. [3] are the immediate compiler antecedent. Their reusable-projection analysis excludes earlier unsuccessful kernel events. Their Section 8.3 reuses corresponding prefix rows,

refreshes after a bounded number of calls, and gives a counter-dependent curve with a first-partial-error term. Our verifier instead resamples indices from a short bank and bounds the fraction of rows with first errors. The distinction is the combined quantitative guarantee and cost regime, not private preprocessing, refresh, or merely removing an age argument from notation. Their Section 9 motivates simultaneous progressive and efficient verification; we do not claim a universal solution for every polynomial modulus.

Taleb–Vergnaud already supply random row sampling and exponential confidence from a code’s distance [2]. Verifier-side caching of their expanded matrix gives a broad derived feasibility baseline without changing ordinary signature generation. Their distance property holds for every residual simultaneously. A bank with $s < n$ cannot have that property; our shorter private bank instead needs secrecy, bounded interaction, refreshing and final precommitment.

Banegas, Le Dévéhat and Smith give private compressed verification for long-lived keys [4]. Their excluded-key estimate in Section 3.4 directly supplies the conditioning principle behind Lemma 4.3, and their security analysis separates source forgeries from compression failure. They target terminal compression. Their inspected statement does not combine the independent short bank, first-spoil fraction tail and fresh progressive sampling used here. We claim no new excluded-key technique, and the seven-row baseline above is our explicitly derived comparator rather than a quoted implementation of their construction.

Slalom precomputes and reuses private Freivalds checks for outsourced computation [5]. It is an important offline/online antecedent, but its inspected layer-integrity theorem is not the progressive bank experiment stated here.

To our knowledge, the combined compiler and quantitative bound in this note have not appeared in the primary statements reviewed. This is a modest, model-specific novelty claim. Its ingredients are familiar, and later identification of an exact antecedent would require revising it. The meaningful claimed advance is the ability to tolerate some previous row errors while retaining a provable progressive curve with an explicit small history term and a verified finite arithmetic crossing.

10 Limitations and next questions

The useful regime needs $Rk + s < q$ and a sufficiently small preparation share W/d . The exact preparation share is $snm/[R(m+n)]$. In the example regime $m \geq n$, making this share small requires R large relative to sn , up to constant factors; $R \gg sn$ is sufficient more generally. In that regime attractive early arithmetic curves consequently need a modulus appreciably larger than skn . We give no uniformly attractive small-modulus construction.

The method is private verifier preprocessing, not public compression or a new signing algorithm. It retains the original matrix for refresh. The finite point is not a certified source-signature hardness parameter set. The source security assumption, hash and norm costs, and any application-specific signature encoding must be supplied independently.

The observable interface is zero/nonzero row bits with ideal finite-field and index randomness. Scalar values, field-operation timing leakage, faults, rollback and key exposure are outside the theorem. Invocations are atomic and serialized. Arbitrary resumable or forkable final sessions require another model. The confidence statement is aggregate/joint, not per-history Bayesian calibration.

Most importantly, the product schedule does not eliminate randomness and zero-fill bursts, allocation costs or memory bandwidth. A trace-erased finite state and exact arithmetic counters are

not compiled code. A concrete implementation, its sampler, a measured workload requiring early confidence, and a full multi-resource comparison are necessary before any practical recommendation.

A Formal artifact and theorem inventory

The source manifest is SOURCE-SHA256SUMS-v2; its SHA-256 digest is

218087b2b1b3961c6a275095e27ca46ec2d42cbb88798732c6f7ea3d7546296d.

The proof entry points below identify the completed trace-free machine wherever possible. Instrumented and erased acceptance masses are proved equal.

Claim	Lean entry points
Uniform-row and adaptive tail	UniformRow; ActualConditionalBound.actual_query_count_bound; ConcreteAdaptiveTail.actual_first_spoil_tail; UniformPoolEndpoint.uniform_pool_bad_count
Aggregate final bound	FinalTraceErasure.final_signature_bound
Joint threshold event	FinalTraceErasure.final_signature_joint
Randomized tapes	VerifiedCompiler.randomized_final_signature_bound; RandomizedFinalJoint.randomized_final_signature_joint, transported through event-generic FinalTraceErasure.finalErrorMass_eq
Perfect completeness	FinalTraceErasure.finalMachine_perfect_completeness
Actual arithmetic	FinalTraceErasure.finalMachine_total_multiplications; CompletedStoppedInvocation.finalMachine_total_multiplications
Epoch and random-draw counts	EpochResourceBounds.coldRun_reservation_epochs; EpochResourceBounds.bankHeadReads_eq; EpochResourceBounds.idealFieldDraws_eq
Trace-free state	TraceErasure.coldRun_refines; FinalTraceErasure.finalMachine_refines; FinalTraceErasure.fieldSlots_card
Progressive interface	VerifierInterface.verifyRequest_isSome; VerifierInterface.requested_confidence_monotone; VerifierInterface.requested_confidence_nontrivial
Source-security separation	StoppedSourceBound.OrdinarySourceBound; StoppedSourceBound.stopped_forgery_bound
Prime field and finite error	PrimeCertificate.parameter_prime_checked; CertifiedField.parameter_field_card; CertifiedParameters.certified_tail_probability; CertifiedParameters.certified_global_error
Matched arithmetic crossing	OperationalEarlyTradeoff.actual_prefix_below_seven_rows; MatchedEarlyCertificates.rs_early_thresholds

Some file names are module names rather than namespaces; the source files give exact fully qualified declarations. The bundle contains no `sorry`, custom axiom or `native_decide`. Primality is checked by bounded divisor certificates, followed by the square-root primality criterion; the tail certificate uses an integer upper bound on the binomial coefficient. The separate clean build used Lean 4.19 and the accompanying mathlib checkout. Its peak observed resident memory was 1,639,044 KiB.

Reproduction. The pinned toolchain is `leanprover/lean4:v4.19.0`; the mathlib checkout is `commit`

`c44e0c8ee63ca166450922a373c7409c5d26b00b`.

Obtain these pinned dependencies and their matching dependency artifacts in a separate build workspace, preserving mathlib’s dependency manifest. From the proof source directory, check `SOURCE-SHA256SUMS-v2`, then run `build-all-v2.sh`. Its ordered input is `modules-v2.txt`; `check.sh` points to the companion `lean-research` toolchain/mathlib layout, takes the shared compile lock, uses one Lean worker and enforces a 120-second and 2,900,000-KiB per-module limit. A source-only replay must remove or quarantine project `.olean` files and rebuild them in that order; merely loading previous compiled artifacts is not that check. The independent replay archive additionally records its exact dependency bootstrap and exhaustive axiom-audit procedure. This paper’s source is independent of the proof directory and compiles with a standard pdfLaTeX installation and the packages listed in its preamble, using two passes to resolve references.

The uniform-bank theorem and operational compiler are fully finite. We do not formalize a concrete instruction set, an entropy-source implementation, a source-signature reduction, or all cited baseline coding results. The resource statements about a conventional dense array’s zero-fill writes are disclosed implementation accounting, not additional kernel-checked machine-code claims.

References

- [1] M. Fischlin. *Progressive Verification: The Case of Message Authentication*. INDOCRYPT, 2003. Extended abstract. https://www.cryptoplexity.informatik.tu-darmstadt.de/media/crypt/publications_1/prog-ver.pdf.
- [2] A. R. Taleb and D. Vergnaud. *Speeding-up verification of digital signatures*. Journal of Computer and System Sciences 116 (2021), 22–39. <https://doi.org/10.1016/j.jcss.2020.08.005>.
- [3] C. Boschini, D. Fiore, E. Pagnin, L. Torresetti and A. Visconti. *Progressive and efficient verification for digital signatures: extensions and experimental results*. Journal of Cryptographic Engineering 14 (2024), 551–575. <https://doi.org/10.1007/s13389-024-00358-0>.
- [4] G. Banegas, A. Le Dévéhat and B. Smith. *Compressed verification for post-quantum signatures with long-term public keys*. arXiv:2509.03098v1, 2025; arXiv record lists CANS 2025. <https://arxiv.org/abs/2509.03098>.
- [5] F. Tramèr and D. Boneh. *Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware*. ICLR, 2019. arXiv:1806.03287v2. <https://arxiv.org/abs/1806.03287>.
- [6] L. H. Nguyen and A. W. Roscoe. *Separating two roles of hashing in one-way message authentication*. FCS-ARSPA-WITS 2008; extended version, Cryptology ePrint Archive, Report 2009/003 (revised 28 November 2009). <https://eprint.iacr.org/2009/003>.